

Федеральное агентство связи

Сибирский государственный университет
телекоммуникаций и информатики

Кафедра ТС и ВС

Лабораторная работа №1

По дисциплине Объектно-ориентированное
программирование

Выполнил: Язиков Денис Александрович

Группа: ИКС-531

Вариант: 24

Проверила: Моренкова Ольга Ильинична

Новосибирск 2026

Лабораторная работа 1

Тема: Простые классы

Цель работы: Научиться создавать класс, его поля, конструктор и методы. Показать чтение и изменение данных объекта.

Задание

Создать класс «Аэропорт» с полями «статус» и «количество полос». Добавить конструктор, методы чтения и изменения полей и вывод всех данных. В main показать объект до и после изменения. Дополнительно используются название аэропорта и город.

Алгоритм решения

1. Создать объект Airport и передать начальные значения в конструктор.
2. Вызвать show и вывести все поля до изменения.
3. Через setName, setCity, setStatus и setRunways изменить каждое поле.
4. Снова вызвать show, затем вызвать все четыре метода get.

Схема алгоритма



Схема отражает последовательность действий программы.

Пояснения к программе

Класс описан в `airport.h`. Поля находятся в `private`, поэтому `main` обращается к ним через методы. Конструктор сразу заполняет все поля.

Методы `get` возвращают значения, методы `set` записывают новые. Метод `show` печатает сведения. `const` у методов чтения означает, что эти методы не меняют объект. Все данные в примере учебные.

Текст программы

lab-01/variant-24/airport.h

```
#ifndef AIRPORT_H
#define AIRPORT_H
#include <iostream>
#include <string>
using namespace std;
class Airport {
private:
    string name;
    string city;
    string status;
    int runways;
public:
    Airport(string n, string c, string s, int r) {
        name = n;
        city = c;
        status = s;
        runways = r;
    }
    string getName() const { return name; }
    string getCity() const { return city; }
    string getStatus() const { return status; }
    int getRunways() const { return runways; }
    void setName(string n) { name = n; }
    void setCity(string c) { city = c; }
    void setStatus(string s) { status = s; }
    void setRunways(int r) { runways = r; }
    void show() const {
        cout << "Название: " << name << endl;
        cout << "Город: " << city << endl;
        cout << "Статус: " << status << endl;
        cout << "Количество полос: " << runways << endl;
    }
};
#endif
```

lab-01/variant-24/airport_basic.cpp

```
#include "airport.h"
int main() {
    Airport airport("Учебный аэропорт", "Новосибирск", "Местный", 1);
    cout << "До изменения:" << endl;
    airport.show();
    airport.setName("Новый учебный аэропорт");
    airport.setCity("Омск");
    airport.setStatus("Международный");
    airport.setRunways(2);
    cout << "\nПосле изменения:" << endl;
    airport.show();
    cout << "\nЧтение полей через методы:" << endl;
    cout << airport.getName() << endl;
    cout << airport.getCity() << endl;
    cout << airport.getStatus() << endl;
    cout << airport.getRunways() << endl;
    return 0;
}
```

Результаты выполнения

Программы собраны с параметрами `-std=c++11 -Wall -Wextra -pedantic`. Предупреждений компилятора нет. Ниже приведены данные контрольного примера и изображения фактического вывода.

Поле	До изменения	После изменения
Название	Учебный аэропорт	Новый учебный аэропорт
Город	Новосибирск	Омск
Статус	Местный	Международный
Полосы	1	2

```
До изменения:
Название: Учебный аэропорт
Город: Новосибирск
Статус: Местный
Количество полос: 1

После изменения:
Название: Новый учебный аэропорт
Город: Омск
Статус: Международный
Количество полос: 2

Чтение полей через методы:
Новый учебный аэропорт
Омск
Международный
2
```

Рисунок 1 - Вывод программы

После вызова методов `set` изменились все четыре поля. Методы `get` вернули те же значения, которые показал `show`. Таким образом, работа всех методов класса проверена.

Ответы на контрольные вопросы

1. Что такое класс и объект

Класс описывает данные и действия. Объект - конкретный экземпляр этого класса.

2. Для чего нужен конструктор

Он вызывается при создании объекта и задаёт начальные значения его полей.

3. Чем отличаются `private` и `public`

Доступ к `private` идёт из методов класса. К `public` можно обращаться из `main` и других частей программы.

Вывод

Создан класс `Airport`. Проверены конструктор, чтение, изменение и вывод всех полей объекта.